🐬 HelPhin LMS — System Architecture & Infrastructure Summary

Dokumen ini menjelaskan arsitektur sistem, teknologi yang digunakan, alasan pemilihan, serta alur kerja deployment HelPhin LMS secara menyeluruh.

> ***Tanggal Deploy:*** *11 April 2026*
> ***Status:*** ✅ *Production — Live at https://lms.helphin.id*

---

1. Gambaran Umum Sistem

HelPhin LMS adalah **Learning Management System** berbasis web yang dirancang untuk mendukung kegiatan pembelajaran digital di lingkungan perguruan tinggi. Sistem ini memungkinkan pengelolaan materi, video pembelajaran, bank soal, latihan soal (Google Form), dan jadwal responsi secara terpusat.

```
┌──────────────────────────────────────────────────────────────┐
│                        USERS (Browser)                       │
│              https://lms.helphin.id                          │
└──────────────────────────────────────────────────────────────┘
                           │
                           │
                    ┌──────▼──────┐
                    │  Cloudflare │  ← DNS & CDN
                    │   (DNS)     │
                    └─────────────┘
                           │
                ┌──────────▼──────────┐
                │      Vercel (Frontend)  │
                │     Next.js 16 (SSR)    │
                │  lms-helphin.vercel.app │
                └─────────────────────┘
                           │ API Calls (HTTPS)
                ┌──────────▼──────────────────┐
                │   Railway (Backend API)      │
                │   Bun + ElysiaJS             │
                │   lmshelphin-production.     │
                │   up.railway.app             │
                │                              │
                │    ┌──────────────────┐      │
                │    │ Railway Volume   │      │
                │    │ /app/uploads     │      │
                │    │ (File Storage)   │      │
                │    └──────────────────┘      │
                └──────────────────────────────┘
                           │ SQL Queries (SSL)
                ┌──────────▼──────────┐
                │   Neon (Database)       │
                │   PostgreSQL            │
                │   ap-southeast-1        │
                └─────────────────────┘
```

---

2. Komponen Sistem & Alasan Pemilihan

2.1 Frontend — Vercel + Next.js 16

Item	Detail
Platform	Vercel
Framework	Next.js 16.1.6 (React 19, Turbopack)
URL Produksi	`https://lms.helphin.id`
URL Cadangan	`https://lms-helphin.vercel.app`
Auto Deploy	✅ Setiap push ke branch `main` di GitHub

Mengapa Vercel?

- **Dibuat oleh tim Next.js** — integrasi paling optimal, zero-config deployment
- **CDN Global** — asset (JS, CSS, gambar) di-cache di edge server terdekat dengan user, sehingga loading sangat cepat
- **Server-Side Rendering (SSR)** — halaman di-render di server untuk SEO dan performa
- **Free tier generous** — cukup untuk skala universitas (100GB bandwidth/bulan)
- **Automatic HTTPS** — SSL certificate otomatis untuk domain custom
- **Preview Deployments** — setiap pull request mendapat URL preview untuk testing sebelum merge

Mengapa Next.js?

- **Full-stack React framework** — routing, SSR, API routes dalam satu framework
- **File-based routing** — struktur folder = struktur URL, mudah dipahami
- **Optimized performance** — automatic code splitting, image optimization, font optimization
- **TypeScript native** — type safety untuk mengurangi bug

2.2 Backend API — Railway + Bun + ElysiaJS

Item	Detail
Platform	Railway
Runtime	Bun 1.3.12
Framework	ElysiaJS 1.4.25
ORM	Drizzle ORM 0.38.4
URL Produksi	`https://lmshelphin-production.up.railway.app`
Root Directory	`/backend`
Build Command	`bun install`
Start Command	`bun run start`
Persistent Storage	Railway Volume mounted di `/app/uploads`

Mengapa Railway?

- **Container-based deployment** — setiap deploy berjalan di container Docker yang terisolasi
- **Persistent Volumes** — file upload tersimpan permanen, tidak hilang saat redeploy
- **Environment Variables** — konfigurasi sensitif (database URL, JWT secret) aman tersimpan
- **Auto-deploy dari GitHub** — push ke `main` = otomatis deploy
- **Monitoring & Logs** — log real-time untuk debugging
- **Free tier** — $5 credit/bulan, cukup untuk development dan small-scale production

Mengapa Bun?

- **3-4x lebih cepat** dari Node.js untuk startup dan execution
- **Built-in bundler & package manager** — tidak perlu npm/yarn terpisah
- **Native TypeScript support** — langsung jalankan `.ts` tanpa compile
- **Drop-in replacement** — kompatibel dengan ekosistem Node.js

Mengapa ElysiaJS?

- **Didesain untuk Bun** — memanfaatkan performa Bun secara maksimal
- **Type-safe** — end-to-end type safety dengan TypeScript
- **Plugin ecosystem** — CORS, JWT, Bearer auth, static file serving tersedia sebagai plugin
- **Ergonomic API** — sintaks clean dan mudah dipahami

Mengapa Drizzle ORM?

- **SQL-like syntax** — query terasa natural bagi developer yang paham SQL
- **Type-safe** — schema TypeScript = tabel database, auto-complete di IDE
- **Zero overhead** — tidak ada runtime proxy, performa mendekati raw SQL
- **Migration system** — `drizzle-kit generate` & `drizzle-kit migrate` untuk versioning schema

2.3 Database — Neon PostgreSQL

Item	Detail
Platform	Neon
Engine	PostgreSQL (serverless)
Region	`ap-southeast-1` (Singapore)
Project	`neondb`
Jumlah Tabel	14 tabel
Connection	SSL/TLS encrypted

Mengapa Neon?

- **Serverless PostgreSQL** — scale to zero saat tidak ada traffic, hemat biaya
- **Branching** — bisa buat "branch" database untuk testing tanpa affect production (mirip Git)
- **Auto-scaling** — compute scale up/down otomatis sesuai beban
- **Region Singapore** — latency rendah untuk user di Indonesia
- **Free tier** — 0.5 GB storage, 190 compute hours/bulan
- **Standard PostgreSQL** — kompatibel penuh, bisa migrasi ke provider lain kapan saja

Struktur Database (14 Tabel):

Tabel	Fungsi
`users`	Data pengguna (admin, dosen, mahasiswa)
`roles`	Definisi role dan permission
`fakultas`	Data fakultas
`prodi`	Data program studi
`mata_kuliah`	Data mata kuliah
`materials`	Materi pembelajaran (PDF, DOC, PPT)
`videos`	Video pembelajaran (YouTube)
`bank_soal`	Arsip bank soal
`exercises`	Latihan soal (Google Form)
`responsi`	Jadwal dan data responsi
`pinned_mata_kuliah`	Mata kuliah yang di-pin user
`material_requests`	Request materi dari mahasiswa
`activity_logs`	Log aktivitas sistem
`otps`	One-time password untuk verifikasi

2.4 DNS & CDN — Cloudflare

Item	Detail
Platform	Cloudflare
Domain	`helphin.id`
Subdomain LMS	`lms.helphin.id` → CNAME ke Vercel
Proxy	DNS Only (non-proxied, agar Vercel SSL berfungsi)

Mengapa Cloudflare?

- **DNS tercepat di dunia** — propagasi dalam hitungan detik
- **DDoS protection** — proteksi bawaan dari serangan DDoS
- **Free SSL** — HTTPS otomatis
- **Analytics** — monitoring traffic domain

Konfigurasi DNS:

Type	Name	Value	Proxy
CNAME	`lms`	`32a8dc9850be6a27.vercel-dns-o17.com`	DNS Only

2.5 Source Code — GitHub

Item	Detail
Repo Utama	`Nurendra-bagaskara/LMS_helphin`
Repo Deploy (Fork)	`TitusTunjungAji/LMS_helphin`
Branch Produksi	`main`

Alur Deploy:

1. Developer push kode ke `TitusTunjungAji/LMS_helphin` branch `main`
2. **Vercel** otomatis detect perubahan di folder `frontend/` → rebuild & deploy frontend
3. **Railway** otomatis detect perubahan di folder `backend/` → rebuild & deploy backend

---

3. Environment Variables

Backend (Railway)

Variable	Fungsi
`DATABASE_URL`	Connection string ke Neon PostgreSQL
`JWT_SECRET`	Secret key untuk signing JWT token autentikasi
`PORT`	Port server (3000)
`NODE_ENV`	Mode environment (production)
`CORS_ORIGIN`	Domain frontend yang diizinkan (`https://lms.helphin.id`)

Frontend (Vercel)

Variable	Fungsi
`NEXT_PUBLIC_API_URL`	URL backend API (`https://lmshelphin-production.up.railway.app`)

---

4. Keamanan

Aspek	Implementasi
HTTPS	✅ Semua traffic terenkripsi (Vercel Auto-SSL + Cloudflare)
CORS	✅ Hanya `https://lms.helphin.id` yang bisa akses API
JWT Authentication	✅ Token-based auth dengan expiry time
Role-Based Access Control	✅ Super Admin, Admin, Mahasiswa dengan permission granular
Database SSL	✅ Koneksi ke Neon menggunakan SSL/TLS
Environment Variables	✅ Secrets disimpan di platform, tidak di source code

---

5. Alur Data (User Flow)

```
User buka lms.helphin.id
        │
        ▼
Cloudflare DNS resolve → Vercel Edge
        │
        ▼
Vercel serve halaman Next.js (SSR/Static)
        │
        ▼
User login → Frontend kirim POST /api/auth/login ke Railway
        │
        ▼
Railway (ElysiaJS) → query ke Neon PostgreSQL
        │
        ▼
Neon return user data → ElysiaJS generate JWT token
        │
        ▼
Frontend simpan token di localStorage
        │
        ▼
Setiap request API → kirim JWT di header Authorization
        │
        ▼
Upload file → disimpan di Railway Volume (/app/uploads)
```

---

6. Estimasi Biaya Bulanan

Service	Plan	Estimasi Biaya
Vercel	Hobby (Free)	**$0**
Railway	Starter ($5 credit)	**~$5/bulan**
Neon	Free Tier	**$0**
Cloudflare	Free	**$0**
GitHub	Free	**$0**
TOTAL		**~~$5/bulan (~~Rp 82.000)**

> ***Catatan:*** *Biaya bisa meningkat jika traffic tinggi. Railway mengenakan biaya berdasarkan penggunaan CPU/RAM. Untuk skala universitas kecil-menengah, budget ini cukup.*

---

7. Akun Default Sistem

Role	Email	Password	Permission

Super Admin	`superadmin@helphin.com`	`superadmin123`	Semua akses (*)
Admin IF	`admin.if@helphin.com`	`admin123`	Manajemen konten prodi
Student IF	`student.if@helphin.com`	`student123`	Akses mahasiswa

> *[!CAUTION]* ***Segera ganti semua password default setelah handover ke tim!***

---

8. Maintenance & Troubleshooting

Cara Cek Status Sistem

- **Frontend:** Buka `https://lms.helphin.id` — kalau tampil = OK
- **Backend:** Buka `https://lmshelphin-production.up.railway.app` — kalau response JSON = OK
- **Database:** Login Railway → View Logs — kalau tidak ada error koneksi = OK

Cara Deploy Update

```
# 1. Edit kode di lokal
# 2. Commit & Push
git add .
git commit -m "feat: deskripsi perubahan"
git push origin main
git push fork main

# 3. Vercel & Railway otomatis rebuild
# 4. Tunggu 1-3 menit, cek status di dashboard masing-masing
```

Cara Jalankan Migration (Tambah/Ubah Tabel)

```
cd backend
export DATABASE_URL="postgresql://neondb_owner:...@neon.tech/neondb?sslmode=require"
bun run db:generate   # Generate migration SQL
bun run db:migrate    # Apply ke database
```

Common Issues

Masalah	Penyebab	Solusi
Frontend blank/error	Build gagal	Cek Vercel → Deployments → Build Logs
API error 500	Backend crash	Cek Railway → Deployments → View Logs
"Forbidden" error	Permission kurang	Login sebagai Super Admin, edit role permission
File upload gagal	Volume penuh	Cek Railway → Volume → Metrics, tambah size
Login gagal	Token expired	Clear localStorage, login ulang

---

9. Rekomendasi Pengembangan Selanjutnya

Prioritas	Item	Deskripsi
🔴 Tinggi	Backup Database	Setup automated backup Neon (daily snapshot)
🔴 Tinggi	Ganti Password Default	Semua akun default harus diganti
🟡 Sedang	Cloud Storage	Migrasi file upload ke Supabase Storage/S3 untuk skalabilitas
🟡 Sedang	Monitoring	Setup uptime monitoring (UptimeRobot/Better Uptime)
🟢 Rendah	Custom Domain Backend	Arahkan `api.helphin.id` ke Railway
🟢 Rendah	CI/CD Pipeline	Setup automated testing sebelum deploy

---

Dokumen ini dibuat pada 11 April 2026. Untuk pertanyaan teknis, hubungi tim development.